

МИНОБРНАУКИ РОССИИ
Санкт-Петербургский государственный
электротехнический университет
«ЛЭТИ» им. В.И. Ульянова (Ленина)
Кафедра САПР

Практическая работа №1
по дисциплине
«Объектно-ориентированное программирование»

Студент гр. 5352 _____

Преподаватель _____

Соболевский М.А.

Кулагин М.В.

Санкт-Петербург

2026

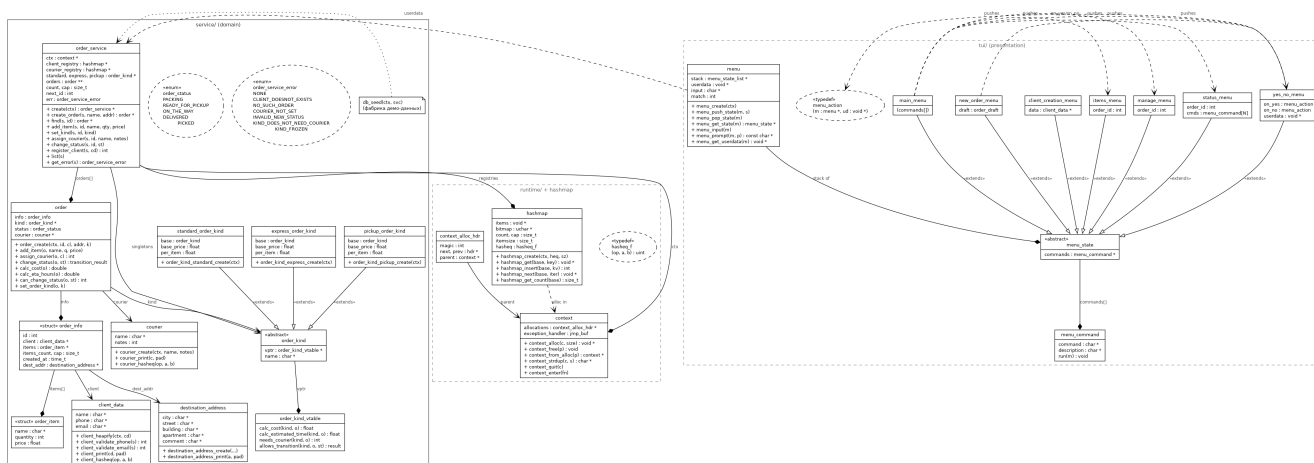
1 Задание

Разработать консольное приложение, которое моделирует работу небольшой службы доставки.

Программа работает только в памяти: внешние файлы, база данных, сетевые запросы и графический интерфейс не являются обязательными.

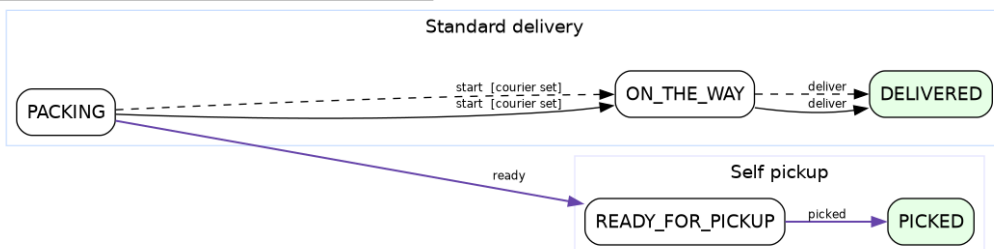
Пользователь должен иметь возможность создать заказ, выбрать способ доставки, назначить курьера, просмотреть список заказов и переводить заказ по допустимым статусам. В программе должны быть предусмотрены корректная обработка ошибочного ввода и понятные сообщения для пользователя.

2 Спецификация программы



Также описание конечного автомата статуса заказов:

Пунктир = та же семантика
 Сплошной = разные переходы по типам
 ANY → INVALID: not in table
 PACKING → ON_THE_WAY: требует courier



3 Описание интерфейса пользователя программы

3.1 Общие сведения

Программа представляет собой интерактивный консольный симулятор службы доставки. Взаимодействие с пользователем построено по схеме *REPL* (read-eval-print loop): после запуска программа переходит в цикл, в котором выводит приглашение `>`, читает строку, введённую пользователем, интерпретирует её как команду и выполняет соответствующее действие. Ввод строки реализован через библиотеку GNU Readline, что даёт пользователю стандартные удобства: редактирование строки, история команд, перемещение по ней стрелками.

Интерфейс организован в виде *стека меню*. В каждый момент времени активно ровно одно меню, у него есть свой набор команд и, при необходимости, собственное состояние (например, черновик заказа или заполняемая карточка клиента). Стек позволяет переходить в подменю (например, из главного меню в меню создания клиента) и возвращаться обратно без потери уже введённых данных.

3.2 Структура меню

В любой момент пользователю доступны:

- команды **текущего меню** (например, у главного меню это `new_order`, `list_orders`, `items`, `manage`, `assign`, `status`);
- **общие команды**, доступные из любого меню: `help` — печать списка команд с описанием, и `quit` / `exit` — завершение работы программы.

При вводе команды, которой нет ни в текущем меню, ни среди общих, программа выводит подсказку:

```
Type "help" for getting all commands for current menu
```

Команда `help` выводит список команд активного меню и общих команд с их кратким описанием. Формат вывода выравнивает названия по максимальной ширине:

```
new_order  create a new order
list_orders list the existing orders
items      add items to an order
manage     choose delivery kind, show cost and ETA
```

<code>assign</code>	<code>assign courier to the order</code>
<code>status</code>	<code>change order status and show current info</code>
<code>help</code>	<code>prints this help message</code>
<code>quit</code>	<code>terminates program</code>
<code>exit</code>	

Завершить работу можно также комбинацией `Ctrl+D` (сигнал EOF на стандартном вводе). В этом случае программа корректно освобождает все ресурсы и завершается с кодом 0.

3.3 Инициализация при запуске

При старте программа автоматически наполняет реестры демонстрационными данными: тремя клиентами (Alice, Bob, Carol) и тремя заказами в разных статусах и с разными способами доставки. Об этом печатается одна строка:

```
[seed] database initialized: 3 clients, 3 orders
```

Это позволяет сразу после запуска опробовать команды `list_orders`, `manage`, `assign` и `status`, не создавая предварительно данные вручную.

3.4 Модель данных

Каждый заказ включает:

- уникальный числовой идентификатор (начинается с 1, монотонно возрастает);
- данные клиента (имя, телефон, электронная почта);
- адрес доставки (город, улица, дом, опционально — квартира и комментарий);
- список позиций (название товара, количество, цена);
- выбранный способ доставки: *Standard delivery*, *Express delivery* или *Self pickup*;
- назначенного курьера (для способов, которые его предусматривают);
- текущий статус;
- время создания, стоимость и ориентировочный срок доставки.

4 Примеры работы и описание

Ниже приведены типовые сценарии работы с программой. Показаны последовательности команд и ответы программы. Строки, начинающиеся с `>`, — это то, что вводит пользователь; остальные строки — вывод программы.

4.1 Запуск и просмотр списка заказов

```
$ ./delivery
[seed] database initialized: 3 clients, 3 orders
> list_orders
Id: 1
Status: Packing
Delivery type: Standard delivery
Created at: Mon Oct 6 12:31:04 2025
ETA: 24 h
Cost: 400.00
Client:
    Name: Alice
    Phone: +1-555-0101
    Email: alice@example.com
Ordered items:
    2 amount of Coffee beans, 12.500000 each
    1 amount of Milk 1L, 3.200000 each
City: Springfield
Street: 742 Evergreen Terrace
Building: 1
-----
Id: 2
Status: On The Way
Delivery type: Express delivery
Created at: Mon Oct 6 12:31:04 2025
ETA: 12 h
Cost: 640.00
Client:
    Name: Bob
    Phone: +1-555-0102
    Email: bob@example.com
Ordered items:
    1 amount of Pizza, 9.900000 each
    2 amount of Cola 0.5L, 1.500000 each
Courier name: Fast Freddie
Courier notes:
    has car
City: Springfield
Street: 123 Fake St
Building: 5B
-----
```

```
Id: 3
Status: Ready For Pickup
Delivery type: Self pickup
Created at: Mon Oct 6 12:31:04 2025
ETA: 1 h
Cost: 0.00
Client:
  Name: Carol
  Phone: +1-555-0103
  Email: carol@example.com
Ordered items:
  1 amount of Book 'C Programming', 42.000000 each
City: Shelbyville
Street: 1 Main St
Building: 10
```

Заказы печатаются по возрастанию идентификатора. Между заказами выводится разделительная линия из дефисов. Информация о курьере печатается, только если он назначен — например, для заказа №3 (самовывоз) курьер не предусмотрен.

4.2 Создание заказа для существующего клиента

Создание заказа — это отдельное меню-визард. Пользователь последовательно задаёт поля черновика, может в любой момент посмотреть текущее состояние и исправить любое из полей.

```
> new_order
> client
client name: Alice
> address
city: Springfield
street: Evergreen Terrace
building: 742
apartment (empty to skip):
comment (empty to skip): Please ring the bell twice
> show
Draft order:
  client:    Alice
  city:      Springfield
  street:    Evergreen Terrace
  building:  742
  apartment: (unset)
  comment:   Please ring the bell twice
> create
Id: 4
Status: Packing
Delivery type: Standard delivery
Created at: Mon Oct 6 12:34:12 2025
```

```
ETA: 24 h
Cost: 300.00
Client:
  Name: Alice
  Phone: +1-555-0101
  Email: alice@example.com
Ordered items:
City: Springfield
Street: Evergreen Terrace
Building: 742
Comment:
  Please ring the bell twice
>
```

После успешного `create` меню автоматически закрывается, программа возвращается в главное меню. Поле `comment` оказалось заполнено, `apartment` — нет, поэтому в блоке адреса выводится только комментарий.

4.3 Создание заказа для нового клиента

Если при создании заказа введено имя клиента, которого нет в реестре, программа предлагает создать новую карточку клиента. Введённые ранее данные заказа при этом не теряются: они сохраняются в черновике, и после регистрации клиента пользователю нужно лишь повторно нажать `create`.

```
> new_order
> client
client name: Dave
> address
city: Ogdenville
street: 12 Elm
building: 3
apartment (empty to skip): 4A
comment (empty to skip):
> create
error: service errored: user does not exists
Client 'Dave' not found. Create new client?
> y
> show
Client:
  name: Dave
  email: (unset)
  phone: (unset)
> email
email: dave@example.com
> phone
phone: +1-555-0200
```

```
> done
Client 'Dave' registered. Type 'create' to finish the order, or 'quit' to exit.
> create
Id: 5
Status: Packing
Delivery type: Standard delivery
Created at: Mon Oct 6 12:36:48 2025
ETA: 24 h
Cost: 300.00
Client:
  Name: Dave
  Phone: +1-555-0200
  Email: dave@example.com
Ordered items:
City: Ogdenville
Street: 12 Elm
Building: 3
Apartment: 4A
>
```

Обратите внимание: поле `name` в карточке клиента уже предзаполнено значением `Dave`, которое пользователь ввёл в черновике заказа — повторно вводить его не нужно.

4.4 Добавление позиций в заказ

`items` открывает меню работы с позициями конкретного заказа. Меню позволяет добавить произвольное количество позиций и завершить работу командой `done`.

```
> items
order id: 4
> add
item name: Espresso beans
quantity: 3
price: 15.75
Item added.
> add
item name: Paper filter
quantity: 1
price: 4.50
Item added.
> done
Id: 4
Status: Packing
Delivery type: Standard delivery
Created at: Mon Oct 6 12:34:12 2025
ETA: 24 h
Cost: 500.00
Client:
  Name: Alice
```

```
Phone: +1-555-0101
Email: alice@example.com
Ordered items:
  3 amount of Espresso beans, 15.750000 each
  1 amount of Paper filter, 4.500000 each
City: Springfield
Street: Evergreen Terrace
Building: 742
Comment:
  Please ring the bell twice
>
```

Стоимость заказа пересчиталась автоматически: после добавления двух позиций стандартная доставка стоит $300 + 50 \times 4 = 500.00$.

4.5 Выбор способа доставки

Меню `manage` позволяет назначить заказу один из трёх способов доставки — `standard`, `express` или `pickup`. После выбора программа печатает пересчитанную стоимость и срок.

```
> manage
order id: 4
> express
Delivery: Express delivery
Cost:      780.00
ETA:      12 h
>
```

Смена способа доставки возможна только до тех пор, пока заказ находится в статусе `Packing`. После начала сборки/отправки попытка сменить способ отклоняется:

```
> manage
order id: 2
error: order kind is frozen after packing started
>
```

4.6 Назначение курьера

Курьер назначается командой `assign`. Программа спрашивает имя курьера и наличие автомобиля. Для самовывоза (`pickup`) курьер не предусмотрен — попытка его назначить завершается ошибкой.

```
> assign
order id: 4
courier name: Speedy Gonzales
```

```
Does the courier have a car?
> y
> list_orders
...
Id: 4
Status: Packing
Delivery type: Express delivery
...
Courier name: Speedy Gonzales
Courier notes:
  has car
...
>
```

```
> assign
order id: 3
courier name: John
Does the courier have a car?
> n
error: service errored: this order kind does not need a courier
>
```

4.7 Изменение статуса заказа

Команда `status` открывает меню статусов. В нём отображаются только те переходы, которые разрешены для текущего способа доставки и текущего состояния заказа. Команда `cancel` закрывает меню без изменений.

```
> status
order id: 4
Id: 4
Status: Packing
Delivery type: Express delivery
...
> on_the_way
Id: 4
Status: On The Way
Delivery type: Express delivery
...
> status
order id: 4
Id: 4
Status: On The Way
Delivery type: Express delivery
...
> delivered
Id: 4
Status: Delivered
```

```
Delivery type: Express delivery
...
>
```

Для самовывоза доступен свой набор переходов: `packing` → `ready_for_pickup` → `picked`. Если попытаться выбрать статус, которого нет в списке команд меню, он просто не будет принят.

4.8 Обработка ошибочного ввода

Программа обрабатывает несколько классов ошибок.

Несуществующий идентификатор заказа.

```
> manage
order id: 42
error: service errored: no such order with the id
>
```

Нечисловой идентификатор.

Функция преобразования возвращает 0, поэтому поиск идёт по идентификатору 0, которого не существует:

```
> status
order id: abc
error: service errored: no such order with the id
>
```

Попытка сменить статус до назначения курьера.

Для стандартной и экспресс-доставки переход в `on_the_way` требует предварительно назначенного курьера:

```
> status
order id: 1
Id: 1
Status: Packing
Delivery type: Standard delivery
...
> on_the_way
error: service errored: courier is not set yet
Id: 1
```

```
Status: Packing
```

```
...  
>
```

Некорректные данные клиента.

При создании карточки клиента программа проверяет email и телефон:

```
> email  
email: notanemail  
> done  
error: invalid email (expected name@domain.tld)  
> phone  
phone: abc  
> done  
error: invalid phone (at least 5 digits, allowed: digits, + - space ( ))  
>
```

Дубликат имени клиента.

```
> name  
name: Alice  
> done  
error: client 'Alice' already exists; pick a different name  
>
```

Пустой черновик заказа.

```
> new_order  
> create  
error: fields 'client', 'city', 'street', 'building' are required  
>
```

Неизвестная команда.

```
> foobar  
Type "help" for getting all commands for current menu  
>
```

4.9 Завершение работы

Выход выполняется командой `quit` (или `exit`), а также комбинацией `Ctrl+D`:

```
> quit
$
```

Перед завершением программа освобождает все выделенные ресурсы. Никаких файлов на диске не создаётся — вся работа ведётся в оперативной памяти.

5 Текст программы

src/service/order.h:

```
1 #ifndef ORDER_H_
2 #define ORDER_H_
3
4 #include "../runtime/context.h"
5 #include "courier.h"
6 #include "client.h"
7 #include "destination_address.h"
8
9 struct order;
10
11 enum order_status_transition_result {
12     OSTR_SUCCESS,
13     OSTR_COURIER_IS_NOT_SET_YET,
14     OSTR_INVALID_NEW_STATUS_FOR_THIS_KIND_OF_ORDER
15 };
16
17 #include "order_kind.h"
18
19 const char *order_status_transition_result_as_cstr(enum order_status_transition_result);
20
21 struct order *order_create(struct context *, int id, client_data *, destination_address *,
22     order_kind *);
23 void order_print(const struct order *);
24 void order_add_item(struct order *o, const char *name, int q, float price);
25 void order_set_order_kind(struct order *, order_kind *);
26 int order_assign_courier(struct order *, courier *);
27 enum order_status_transition_result order_change_status(struct order *, enum order_status
28     new_status);
29
30 int order_get_id(struct order const *);
31 size_t order_get_items_count(const struct order *);
32 enum order_status order_get_status(const struct order *);
33 courier *order_get_courier(const struct order *);
34
35 double order_calc_cost(const struct order *);
36 double order_calc_eta_hours(const struct order *);
37 int order_can_change_status(const struct order *, enum order_status);
38 int order_can_change_kind(const struct order *);
39 const char *order_status_as_cstr(enum order_status);
40
41 #endif /* ORDER_H_ */
```

src/service/courier.h:

```

1 #ifndef COURIER_H_
2 #define COURIER_H_
3
4 #include "../runtime/context.h"
5 #include "../hashmap.h"
6
7 enum {
8     COURIER_NOTES_NONE      = 0x0,
9     COURIER_NOTES_HAS_CAR   = 0x1
10 };
11
12 struct courier;
13
14 typedef struct courier const courier;
15
16 courier *courier_create(struct context *, const char *name, int);
17 void courier_print(courier *, int pad);
18 unsigned courier_hasheq(enum hasheq_op, const void *, const void *);
19
20 #endif /* COURIER_H_ */

```

src/service/destination_address.h:

```

1 #ifndef DESTINATION_ADDRESS_H_
2 #define DESTINATION_ADDRESS_H_
3
4 #include "../runtime/context.h"
5
6 struct destination_address {
7     const char *city;
8     const char *street;
9     const char *building;
10    const char *apartment;
11    const char *comment;
12 };
13
14 typedef const struct destination_address destination_address;
15
16 destination_address *destination_address_create(struct context *ctx, const char *city,
17     const char *street, const char *building, const char *apartment, const char *comment);
18 void destination_address_print(destination_address *, int pad);
19 #endif /* DESTINATION_ADDRESS_H_ */

```

src/service/order_kind.h:

```

1 #ifndef ORDER_KIND_H_
2 #define ORDER_KIND_H_
3
4 struct order_kind;
5 struct order;
6
7 #include "order.h"
8
9 enum order_status {
10     ORDER_STATUS_PACKING,
11     ORDER_STATUS_READY_FOR_PICKUP,

```

```

12     ORDER_STATUS_ON_THE_WAY,
13     ORDER_STATUS_DELIVERED,
14     ORDER_STATUS_PICKED
15 };
16
17 struct allowed_order_status_transition {
18     enum order_status from, to;
19 };
20
21 typedef const struct order_kind order_kind;
22
23 #define ORDER_KIND_METHODS \
24     M(float, calc_cost, (kind, order), (order_kind *, const struct order *)) \
25     M(float, calc_estimated_time, (kind, order), (order_kind *, const struct order *)) \
26     M(int, needs_courier, (kind, order), (order_kind *, const struct order *)) \
27     M(enum order_status_transition_result, allows_transition, (kind, order, status),
28         (order_kind *, const struct order *, enum order_status)) \
29
30 struct order_kind_vtable {
31 #define M(ret, name, pnames, ptypes) ret (*name)ptypes;
32     ORDER_KIND_METHODS
33 #undef M
34 };
35
36 struct order_kind {
37     const struct order_kind_vtable *vptr;
38     const char *name;
39 };
40
41 order_kind *order_kind_standard_create(struct context *ctx);
42 order_kind *order_kind_express_create(struct context *ctx);
43 order_kind *order_kind_pickup_create(struct context *ctx);
44 #endif /* ORDER_KIND_H_ */

```

src/service/db.h:

```

1 #ifndef DB_H_
2 #define DB_H_
3
4 #include "../runtime/context.h"
5 #include "order_service.h"
6
7 /* Заполняет реестры сервиса стартовым набором клиентов и заказов.
8    Вызывается один раз после order_service_create(). */
9 void db_seed(struct context *ctx, struct order_service *svc);
10
11 #endif /* DB_H_ */

```

src/service/client.h:

```

1 #ifndef CLIENT_H_
2 #define CLIENT_H_
3
4 #include "../hashmap.h"
5 #include "../runtime/context.h"
6
7 struct client_data {

```

```

8   const char *name;
9   const char *phone;
10  const char *email;
11 };
12
13 typedef const struct client_data client_data;
14
15 client_data *client_heapify(struct context *, client_data *);
16 int client_validate_phone(const char *phone);
17 int client_validate_email(const char *email);
18 void client_print(client_data *, int pad);
19 unsigned client_hasheq(enum hasheq_op, const void *, const void *);
20
21 #endif /* CLIENT_H_ */

```

src/service/order_service.h:

```

1  #ifndef ORDER_SERVICE_H_
2  #define ORDER_SERVICE_H_
3
4  #include "order.h"
5
6  enum order_service_error {
7      OSE_NONE,
8      OSE_CLIENT_DOESNOT_EXISTS,
9      OSE_NO_SUCH_ORDER_WITH_ID,
10     OSE_COURIER_IS_NOT_SET_YET,
11     OSE_INVALID_NEW_STATUS,
12     OSE_KIND_DOES_NOT_NEED_COURIER,
13     OSE_KIND_FROZEN
14 };
15
16 struct order_service *order_service_create(struct context *ctx);
17 struct order *order_service_create_order(struct order_service *s, const char *client_name,
18     destination_address *);
19 struct order *order_service_find(struct order_service *s, int id);
20 int order_service_register_client(struct order_service *s, client_data *);
21 void order_service_list(struct order_service *s);
22 void order_service_set_kind(struct order_service *s, int id, order_kind *);
23 void order_service_assign_courier(struct order_service *s, int id, const char *name, int
24     notes);
25 void order_service_change_status(struct order_service *s, int id, enum order_status st);
26 void order_service_add_item(struct order_service *s, int id, const char *name, int qty,
27     float price);
28 enum order_service_error order_service_get_error(struct order_service *s);
29
30 order_kind *order_service_get_standard_kind(struct order_service *s);
31 order_kind *order_service_get_express_kind(struct order_service *s);
32 order_kind *order_service_get_pickup_kind(struct order_service *s);
33
34 #endif /* ORDER_SERVICE_H_ */

```

src/tui/menu.h:

```

1  #ifndef MENU_H_
2  #define MENU_H_
3
4  #include "../runtime/context.h"

```

```

5
6 struct menu;
7
8 struct menu_state;
9
10 struct menu_command {
11     const char *command;
12     const char *description;
13     void (*run)(struct menu *);
14 };
15
16 struct menu_state {
17     const struct menu_command *commands;
18 };
19
20 typedef void (*menu_action)(struct menu *, void *);
21
22 struct menu *menu_create(struct context *);
23 void menu_input(struct menu *m);
24 void menu_push_state(struct menu *m, struct menu_state *);
25 void menu_pop_state(struct menu *m);
26 struct menu_state *menu_get_state(struct menu *m);
27 void *menu_get_userdata(struct menu *m);
28 void menu_set_userdata(struct menu *m, void *);
29 const char *menu_prompt(struct menu *m, const char *prompt);
30
31 const char *menu_get_input(struct menu *m);
32
33 #endif /* MENU_H_ */

```

src/tui/menus.h:

```

1 #ifndef MENUS_H_
2 #define MENUS_H_
3
4 #include "../runtime/context.h"
5 #include "menu.h"
6 #include "../service/client.h"
7
8 struct order;
9
10 struct menu_state *yes_no_menu_create(struct context *ctx,
11                                     menu_action on_yes,
12                                     menu_action on_no,
13                                     void *userdata);
14 struct menu_state *client_creation_menu_create(struct context *ctx, const struct
15 client_data *initial);
16 struct menu_state *main_menu_create(struct context *);
17 struct menu_state *new_order_menu_create(struct context *ctx);
18 struct menu_state *manage_menu_create(struct context *ctx, int order_id);
19 struct menu_state *status_menu_create(struct context *ctx, int order_id, struct order *o);
20 struct menu_state *items_menu_create(struct context *ctx, int order_id);
21 #endif /* MENUS_H_ */

```

src/tui/errors.h:

```

1 #ifndef TUI_ERRORS_H_

```

```

2 | #define TUI_ERRORS_H_
3 |
4 | #include "../service/order_service.h"
5 |
6 | void print_order_service_error(enum order_service_error err);
7 |
8 | #endif /* TUI_ERRORS_H_ */

```

src/runtime/context.h:

```

1 | #ifndef CONTEXT_H_
2 | #define CONTEXT_H_
3 |
4 | #include <stddef.h>
5 |
6 | struct context;
7 |
8 | enum context_exception {
9 |     CTX_E_NONE,
10 |    CTX_E_QUIT,
11 |    CTX_E_MALLOC_FAILURE
12 | };
13 |
14 | void context_enter(void (*)(struct context *));
15 | void context_quit(struct context *c);
16 | void *context_alloc(struct context *c, size_t size);
17 | char *context_strdup(struct context *c, const char *);
18 | void context_free(void *);
19 | struct context *context_from_alloc(void *);
20 |
21 | #endif /* CONTEXT_H_ */

```

src/hashmap.h:

```

1 | #ifndef HASHMAP_H_
2 | #define HASHMAP_H_
3 |
4 | #include "runtime/context.h"
5 |
6 | enum hasheq_op {
7 |     HASHEQ_HASH,
8 |     HASHEQ_EQ
9 | };
10 |
11 | struct hashmap;
12 |
13 | typedef unsigned int (*hasheq_f)(enum hasheq_op, const void *, const void *);
14 |
15 | struct hashmap *hashmap_create(struct context *ctx, hasheq_f, size_t itemsz);
16 | void *hashmap_get(struct hashmap *, const void *);
17 | int  hashmap_insert(struct hashmap *, const void *); /* = 1 if element is new - count is
    modified */
18 | void *hashmap_next(struct hashmap *, size_t *iter);
19 | size_t hashmap_get_count(struct hashmap *);
20 | unsigned fnv1(const void *, size_t);
21 |
22 | #endif /* HASHMAP_H_ */

```

src/main.c:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #include "runtime/context.h"
6 #include "service/db.h"
7 #include "service/order_service.h"
8 #include "tui/menus.h"
9
10 void run(struct context *ctx)
11 {
12     struct menu *menu = menu_create(ctx);
13     struct order_service *svc = order_service_create(ctx);
14
15     db_seed(ctx, svc);
16
17     menu_set_userdata(menu, svc);
18     menu_push_state(menu, main_menu_create(ctx));
19     for (;;) menu_input(menu);
20 }
21
22 int main(void)
23 {
24     context_enter(run);
25     return 0;
26 }
```

src/service/order_service.c:

```
1 #include "order_service.h"
2
3 #include <assert.h>
4 #include <stdlib.h>
5 #include <string.h>
6 #include <stdio.h>
7
8 #include "../hashmap.h"
9
10 struct client_registry {
11     struct hashmap *map;
12 };
13
14 struct courier_registry {
15     struct hashmap *map;
16 };
17
18 struct order_service {
19     struct context *ctx;
20
21     struct client_registry client_registry;
22     struct courier_registry courier_registry;
23
24     /* Singletons */
25     order_kind *standard;
26     order_kind *express;
27     order_kind *pickup;
28 }
```

```

29     struct order **orders;
30     size_t count, cap;
31     int next_id;
32
33     enum order_service_error err;
34 };
35
36 enum order_service_error order_service_get_error(struct order_service *s)
37 {
38     return s->err;
39 }
40
41 struct order *order_service_find(struct order_service *s, int id)
42 {
43     size_t mid, lo, hi;
44
45     s->err = OSE_NONE;
46     lo = 0;
47     hi = s->count;
48
49     while (lo < hi) {
50         int c_id;
51         mid = lo + (hi - lo)/2;
52         c_id = order_get_id(s->orders[mid]);
53         if (c_id < id) lo = mid + 1;
54         else if (c_id > id) hi = mid;
55         else return s->orders[mid];
56     }
57     s->err = OSE_NO_SUCH_ORDER_WITH_ID;
58     return NULL;
59 }
60
61 struct order_service *order_service_create(struct context *ctx)
62 {
63     struct order_service *svc;
64     svc = context_alloc(ctx, sizeof *svc);
65     svc->ctx = ctx;
66     /* this is stupid, but the data should not move, so its kinda makes sense */
67     svc->client_registry.map = hashmap_create(ctx, client_hasheq, sizeof(client_data*));
68     svc->courier_registry.map = hashmap_create(ctx, courier_hasheq, sizeof(courier*));
69     svc->standard = order_kind_standard_create(ctx);
70     svc->express = order_kind_express_create(ctx);
71     svc->pickup = order_kind_pickup_create(ctx);
72     svc->next_id = 1;
73     return svc;
74 }
75
76 void order_service_set_kind(struct order_service *s, int id, order_kind *kind)
77 {
78     struct order *order = order_service_find(s, id);
79     if (!order) return;
80
81     if (!order_can_change_kind(order)) {
82         s->err = OSE_KIND_FROZEN;
83         return;
84     }
85     order_set_order_kind(order, kind);
86 }

```

```

87
88 void order_service_assign_courier(struct order_service *s, int id, const char *name, int
    notes)
89 {
90     struct order *order = order_service_find(s, id);
91     courier *c;
92     if (!order) return;
93
94     c = courier_create(s->ctx, name, notes);
95     if (!order_assign_courier(order, c)) {
96         s->err = OSE_KIND_DOES_NOT_NEED_COURIER;
97         return;
98     }
99 }
100
101 void order_service_change_status(struct order_service *s, int id, enum order_status st)
102 {
103     struct order *order = order_service_find(s, id);
104     if (!order) return;
105
106     switch (order_change_status(order, st)) {
107     case OSTR_SUCCESS: break;
108     case OSTR_COURIER_IS_NOT_SET_YET:
109         s->err = OSE_COURIER_IS_NOT_SET_YET;
110         break;
111     case OSTR_INVALID_NEW_STATUS_FOR_THIS_KIND_OF_ORDER:
112         s->err = OSE_INVALID_NEW_STATUS;
113         break;
114     }
115 }
116
117 void order_service_add_item(struct order_service *s, int id, const char *name, int qty,
    float price)
118 {
119     struct order *o = order_service_find(s, id);
120     if (!o) return;
121     order_add_item(o, name, qty, price);
122 }
123
124 void order_service_add_order(struct order_service *s, struct order *order)
125 {
126     if (s->count >= s->cap) {
127         void *new_orders;
128         if (s->cap) s->cap *= 2;
129         else s->cap = 1;
130         new_orders = context_alloc(s->ctx, sizeof *s->orders * s->cap);
131         memcpy(new_orders, s->orders, sizeof *s->orders * s->count);
132         context_free(s->orders);
133         s->orders = new_orders;
134     }
135
136     s->orders[s->count++] = order;
137 }
138
139 client_data *order_service_find_client(struct client_registry *reg, const char *client_name)
140 {
141     struct client_data key = {0};
142     client_data *const *c;

```

```

143     struct client_data *pkey;
144     key.name = client_name;
145     pkey = &key;
146     c = hashmap_get(reg->map, &pkey);
147     if (!c) return NULL;
148     return *c;
149 }
150
151 struct order *order_service_create_order(struct order_service *s, const char *client_name,
    destination_address *addr)
152 {
153     struct order *order;
154     client_data *client;
155
156     client = order_service_find_client(&s->client_registry, client_name);
157     if (!client) {
158         s->err = OSE_CLIENT_DOESNOT_EXISTS;
159         return NULL;
160     }
161
162     order = order_create(s->ctx, s->next_id++, client, addr, s->standard);
163     order_service_add_order(s, order);
164
165     return order;
166 }
167
168 void order_service_list(struct order_service *s)
169 {
170     size_t i;
171     for (i = 0; i < s->count; ++i) {
172         if (i) printf("-----\n");
173         order_print(s->orders[i]);
174     }
175 }
176
177 int order_service_register_client(struct order_service *s, client_data *cd)
178 {
179     client_data *d = order_service_find_client(&s->client_registry, cd->name);
180     if (d) return 1;
181     hashmap_insert(s->client_registry.map, &cd);
182     return 0;
183 }
184
185 order_kind *order_service_get_standard_kind(struct order_service *s)
186 {
187     return s->standard;
188 }
189
190 order_kind *order_service_get_express_kind(struct order_service *s)
191 {
192     return s->express;
193 }
194
195 order_kind *order_service_get_pickup_kind(struct order_service *s)
196 {
197     return s->pickup;
198 }

```

src/service/courier.c:

```
1 #include "courier.h"
2
3 #include <stdio.h>
4 #include <string.h>
5 #include <assert.h>
6
7 struct courier {
8     const char *name;
9     int notes;
10 };
11
12 courier *courier_create(struct context *ctx, const char *name, int notes)
13 {
14     struct courier *c = context_alloc(ctx, sizeof *c);
15     c->name = name;
16     c->notes = notes;
17     return c;
18 }
19
20 void courier_print(courier *c, int pad)
21 {
22     printf("%sCourier name: %s\n", pad, "", c->name);
23     printf("%sCourier notes:\n", pad, "");
24     if (c->notes & COURIER_NOTES_HAS_CAR)
25         printf("%shas car\n", pad+2, "");
26 }
27
28 unsigned courier_hasheq(enum hasheq_op op, const void *pa, const void *pb)
29 {
30     courier *const *a, *const *b;
31     a = pa;
32     b = pb;
33     switch (op) {
34     case HASHEQ_EQ:
35         return strcmp((*a)->name, (*b)->name) == 0;
36     case HASHEQ_HASH:
37         return fnv1((*a)->name, strlen((*a)->name));
38     }
39     return -1;
40 }
```

src/service/db.c:

```
1 #include "db.h"
2
3 #include <stdio.h>
4
5 #include "order.h"
6 #include "client.h"
7 #include "courier.h"
8 #include "destination_address.h"
9 #include "order_kind.h"
10
11 /* Хелпер: собрать client_data в арене и зарегистрировать в сервисе.
12    Строки - литералы (.rodata), живут столько же, сколько процесс,
13    поэтому их копировать в арену не нужно. */
14 static void seed_client(struct context *ctx,
```

```

15         struct order_service *svc,
16         const char *name,
17         const char *phone,
18         const char *email)
19 {
20     struct client_data tmp;
21     client_data *c;
22
23     tmp.name = name;
24     tmp.phone = phone;
25     tmp.email = email;
26
27     c = client_heapify(ctx, &tmp);
28     if (order_service_register_client(svc, c) != 0)
29         fprintf(stderr, "[seed] duplicate client skipped: %s\n", name);
30 }
31
32 static destination_address *seed_address(struct context *ctx,
33                                         const char *city,
34                                         const char *street,
35                                         const char *building)
36 {
37     return destination_address_create(ctx, city, street, building, NULL, NULL);
38 }
39
40 struct seed_item {
41     const char *name;
42     int quantity;
43     float price;
44 };
45
46 static struct order *seed_order(struct order_service *svc,
47                                 const char *client,
48                                 destination_address *addr,
49                                 order_kind *kind,
50                                 const struct seed_item *items,
51                                 size_t items_n)
52 {
53     struct order *o;
54     size_t i;
55     int id;
56
57     o = order_service_create_order(svc, client, addr);
58     if (!o) {
59         fprintf(stderr, "[seed] cannot create order for %s\n", client);
60         return NULL;
61     }
62
63     id = order_get_id(o);
64     order_service_set_kind(svc, id, kind);
65
66     for (i = 0; i < items_n; ++i)
67         order_service_add_item(svc, id, items[i].name,
68                               items[i].quantity, items[i].price);
69
70     return o;
71 }
72

```

```

73 void db_seed(struct context *ctx, struct order_service *svc)
74 {
75     struct order *o;
76     int id;
77
78     /* --- Clients ----- */
79     seed_client(ctx, svc, "Alice", "+1-555-0101", "alice@example.com");
80     seed_client(ctx, svc, "Bob", "+1-555-0102", "bob@example.com");
81     seed_client(ctx, svc, "Carol", "+1-555-0103", "carol@example.com");
82
83     /* --- Order 1: Alice, Standard, PACKING ----- */
84     {
85         static const struct seed_item items[] = {
86             {"Coffee beans", 2, 12.50f},
87             {"Milk 1L", 1, 3.20f},
88         };
89         destination_address *addr =
90             seed_address(ctx, "Springfield", "742 Evergreen Terrace", "1");
91         seed_order(svc, "Alice", addr, order_service_get_standard_kind(svc),
92             items, sizeof items / sizeof items[0]);
93     }
94
95     /* --- Order 2: Bob, Express, ON_THE_WAY (courier assigned) ----- */
96     {
97         static const struct seed_item items[] = {
98             {"Pizza", 1, 9.90f},
99             {"Cola 0.5L", 2, 1.50f},
100         };
101         destination_address *addr =
102             seed_address(ctx, "Springfield", "123 Fake St", "5B");
103
104         o = seed_order(svc, "Bob", addr, order_service_get_express_kind(svc),
105             items, sizeof items / sizeof items[0]);
106         if (o) {
107             id = order_get_id(o);
108             order_service_assign_courier(svc, id, "Fast Freddie",
109                 COURIER_NOTES_HAS_CAR);
110             order_service_change_status(svc, id, ORDER_STATUS_ON_THE_WAY);
111         }
112     }
113
114     /* --- Order 3: Carol, Pickup, READY_FOR_PICKUP ----- */
115     {
116         static const struct seed_item items[] = {
117             {"Book 'C Programming'", 1, 42.00f},
118         };
119         destination_address *addr =
120             seed_address(ctx, "Shelbyville", "1 Main St", "10");
121
122         o = seed_order(svc, "Carol", addr, order_service_get_pickup_kind(svc),
123             items, sizeof items / sizeof items[0]);
124         if (o) {
125             id = order_get_id(o);
126             order_service_change_status(svc, id, ORDER_STATUS_READY_FOR_PICKUP);
127         }
128     }
129
130     printf("[seed] database initialized: 3 clients, 3 orders\n");

```

131 }

src/service/order_kinds/express.c:

```
1 #include "../order.h"
2
3 #include <assert.h>
4 #include <stdio.h>
5
6 #define M(ret, name, pnames, ptypes) static ret exp_##name ptypes;
7 ORDER_KIND_METHODS
8 #undef M
9
10 static const struct order_kind_vtable vtable = {
11 #define M(ret, name, pnames, ptypes) exp_##name,
12 ORDER_KIND_METHODS
13 #undef M
14 };
15
16 struct express_order_kind {
17     struct order_kind base;
18     float base_price;
19     float per_item;
20 };
21
22 order_kind *order_kind_express_create(struct context *ctx)
23 {
24     struct express_order_kind *s;
25     s = context_alloc(ctx, sizeof *s);
26     s->base.vptr = &vtable;
27     s->base.name = "Express delivery";
28     s->base_price = 500.0f;
29     s->per_item = 70.0f;
30     return &s->base;
31 }
32
33 static float exp_calc_cost(order_kind *kind, const struct order *o)
34 {
35     const struct express_order_kind *s = (const void *)kind;
36     return s->base_price + s->per_item * order_get_items_count(o);
37 }
38
39 static float exp_calc_estimated_time(order_kind *kind, const struct order *o)
40 {
41     (void) kind;
42     (void) o;
43     return 12;
44 }
45
46
47 static int exp_needs_courier(order_kind *kind, const struct order *o)
48 {
49     (void) kind;
50     (void) o;
51     return 1;
52 }
53
54 static enum order_status_transition_result exp_allows_transition(order_kind *kind, const
```

```

55 struct order *order, enum order_status to)
56 {
57     static const struct allowed_order_status_transition table[] = {
58         {ORDER_STATUS_PACKING, ORDER_STATUS_ON_THE_WAY},
59         {ORDER_STATUS_ON_THE_WAY, ORDER_STATUS_DELIVERED},
60         {-1, -1}
61     };
62     struct allowed_order_status_transition const *tp = table;
63
64     enum order_status from = order_get_status(order);
65
66     (void) kind;
67
68     for (; (int)tp->from != -1; ++tp) {
69         if (tp->from != from) continue;
70         if (tp->to != to) continue;
71
72         switch (from) {
73             case ORDER_STATUS_PACKING:
74                 if (!order_get_courier(order)) return OSTR_COURIER_IS_NOT_SET_YET;
75             default:
76                 break;
77         }
78
79         return OSTR_SUCCESS;
80     }
81     return OSTR_INVALID_NEW_STATUS_FOR_THIS_KIND_OF_ORDER;
82 }

```

src/service/order_kinds/standard.c:

```

1 #include "../order.h"
2
3 #include <assert.h>
4
5 #define M(ret, name, pnames, ptypes) static ret std_##name ptypes;
6 ORDER_KIND_METHODS
7 #undef M
8
9 static const struct order_kind_vtable vtable = {
10 #define M(ret, name, pnames, ptypes) std_##name,
11 ORDER_KIND_METHODS
12 #undef M
13 };
14
15 struct standard_order_kind {
16     struct order_kind base;
17     float base_price;
18     float per_item;
19 };
20
21 order_kind *order_kind_standard_create(struct context *ctx)
22 {
23     struct standard_order_kind *s;
24     s = context_alloc(ctx, sizeof *s);
25     s->base.vp_ptr = &vtable;
26     s->base.name = "Standard delivery";

```

```

27     s->base_price = 300.0f;
28     s->per_item = 50.0f;
29     return &s->base;
30 }
31
32 static float std_calc_cost(order_kind *kind, const struct order *o)
33 {
34     const struct standard_order_kind *s = (const void *)kind;
35     return s->base_price + s->per_item * order_get_items_count(o);
36 }
37
38 static float std_calc_estimated_time(order_kind *kind, const struct order *o)
39 {
40     (void)o;
41     (void)kind;
42     return 24;
43 }
44
45 static int std_needs_courier(order_kind *kind, const struct order *o)
46 {
47     (void)o;
48     (void)kind;
49     return 1;
50 }
51
52 static enum order_status_transition_result
53 std_allows_transition(order_kind *kind, const struct order *order, enum order_status to)
54 {
55     static const struct allowed_order_status_transition table[] = {
56         {ORDER_STATUS_PACKING, ORDER_STATUS_ON_THE_WAY},
57         {ORDER_STATUS_ON_THE_WAY, ORDER_STATUS_DELIVERED},
58         {-1, -1}
59     };
60
61     struct allowed_order_status_transition const *tp;
62     enum order_status from;
63
64     (void)kind;
65
66     tp = table;
67     from = order_get_status(order);
68
69     for (; (int)tp->from != -1; ++tp) {
70         if (tp->from != from) continue;
71         if (tp->to != to) continue;
72
73         if (from == ORDER_STATUS_PACKING) {
74             if (!order_get_courier(order)) return OSTR_COURIER_IS_NOT_SET_YET;
75         }
76
77         return OSTR_SUCCESS;
78     }
79     return OSTR_INVALID_NEW_STATUS_FOR_THIS_KIND_OF_ORDER;
80 }

```

src/service/order_kinds/pickup.c:

```

1 #include "../order.h"

```

```

2
3 #include <assert.h>
4
5 #define M(ret, name, pnames, ptypes) static ret pkp_##name ptypes;
6 ORDER_KIND_METHODS
7 #undef M
8
9 static const struct order_kind_vtable vtable = {
10 #define M(ret, name, pnames, ptypes) pkp_##name,
11 ORDER_KIND_METHODS
12 #undef M
13 };
14
15 struct pickup_order_kind {
16     struct order_kind base;
17     float base_price;
18     float per_item;
19 };
20
21 order_kind *order_kind_pickup_create(struct context *ctx)
22 {
23     struct pickup_order_kind *s;
24     s = context_alloc(ctx, sizeof *s);
25     s->base.vptr = &vtable;
26     s->base.name = "Self pickup";
27     s->base_price = 0.0f;
28     s->per_item = 0.0f;
29     return &s->base;
30 }
31
32 static float pkp_calc_cost(order_kind *kind, const struct order *o)
33 {
34     const struct pickup_order_kind *s = (const void *)kind;
35     return s->base_price + s->per_item * order_get_items_count(o);
36 }
37
38 static float pkp_calc_estimated_time(order_kind *kind, const struct order *o)
39 {
40     (void)kind;
41     (void)o;
42     return 1;
43 }
44
45 static int pkp_needs_courier(order_kind *kind, const struct order *o)
46 {
47     (void)kind;
48     (void)o;
49     return 0;
50 }
51
52 static enum order_status_transition_result
53 pkp_allows_transition(order_kind *kind, const struct order *order, enum order_status to)
54 {
55     static const struct allowed_order_status_transition table[] = {
56         {ORDER_STATUS_PACKING, ORDER_STATUS_READY_FOR_PICKUP},
57         {ORDER_STATUS_READY_FOR_PICKUP, ORDER_STATUS_PICKED},
58         {-1, -1}
59     };

```

```

60
61     struct allowed_order_status_transition const *tp;
62     enum order_status from;
63
64     (void)kind;
65
66     from = order_get_status(order);
67
68     for (tp = table; (int)tp->from != -1; ++tp) {
69         if (tp->from != from) continue;
70         if (tp->to != to) continue;
71         return OSTR_SUCCESS;
72     }
73     return OSTR_INVALID_NEW_STATUS_FOR_THIS_KIND_OF_ORDER;
74 }

```

src/service/order.c:

```

1  #include <time.h>
2  #include <assert.h>
3  #include <stdio.h>
4  #include <string.h>
5
6  #include "order.h"
7
8  struct order_item {
9      const char *name;
10     int quantity;
11     float price;
12 };
13
14 struct order_info {
15     int id;
16     client_data *client;
17     struct order_item *items;
18     size_t items_count;
19     size_t items_cap;
20     time_t created_at;
21     destination_address *dest_addr;
22 };
23
24 struct order {
25     struct order_info info;
26     order_kind *kind;
27     enum order_status status;
28     courier *courier;
29 };
30
31 struct order *order_create(struct context *ctx, int id, client_data *client,
32     destination_address *addr, order_kind *kind)
33 {
34     struct order *o = context_alloc(ctx, sizeof *o);
35     o->info.id = id;
36     o->info.client = client;
37     o->info.dest_addr = addr;
38     o->kind = kind;
39     time(&o->info.created_at);
40     return o;

```

```

40 }
41
42 const char *order_status_as_cstr(enum order_status status)
43 {
44     switch (status) {
45         case ORDER_STATUS_PACKING:         return "Packing";
46         case ORDER_STATUS_READY_FOR_PICKUP: return "Ready For Pickup";
47         case ORDER_STATUS_ON_THE_WAY:      return "On The Way";
48         case ORDER_STATUS_DELIVERED:       return "Delivered";
49         case ORDER_STATUS_PICKED:          return "Picked";
50     }
51     assert(0 && "unreachable");
52 }
53
54 void order_add_item(struct order *order, const char *name, int q, float price)
55 {
56     struct context *c = context_from_alloc(order);
57     struct order_item item;
58     struct order_info *info = &order->info;
59
60     if (info->items_count >= info->items_cap) {
61         void *new_items;
62         if (info->items_cap) info->items_cap *= 2;
63         else info->items_cap = 1;
64         new_items = context_alloc(c, sizeof *info->items * info->items_cap);
65         memcpy(new_items, info->items, sizeof *info->items * info->items_count);
66         context_free(info->items);
67         info->items = new_items;
68     }
69
70     item.name = name;
71     item.quantity = q;
72     item.price = price;
73
74     info->items[info->items_count++] = item;
75 }
76
77 static void print_order_item(struct order_item const *item, int pad)
78 {
79     printf("%*s%d amount of %s, %f each\n", pad, "", item->quantity, item->name,
80         item->price);
81 }
82
83 void order_print(const struct order *o)
84 {
85     size_t i;
86     const char *time;
87
88     printf("Id: %d\n", o->info.id);
89     printf("Status: %s\n", order_status_as_cstr(o->status));
90     printf("Delivery type: %s\n", o->kind->name);
91     time = ctime(&o->info.created_at);
92     printf("Created at: %.*s\n", (int)strlen(time)-1, time);
93     printf("ETA: %.0f h\n", order_calc_eta_hours(o));
94     printf("Cost: %.2f\n", order_calc_cost(o));
95     printf("Client:\n");
96     client_print(o->info.client, 4);
97     printf("Ordered items:\n");

```

```

97     for (i = 0; i < o->info.items_count; ++i) {
98         print_order_item(&o->info.items[i], 4);
99     }
100     if (o->courier) courier_print(o->courier, 0);
101     destination_address_print(o->info.dest_addr, 0);
102 }
103
104 int order_assign_courier(struct order *o, courier *c)
105 {
106     if (!o->kind->vptr->needs_courier(o->kind, o)) return 0;
107     o->courier = c;
108     return 1;
109 }
110
111 const char *order_status_transition_result_as_cstr(enum order_status_transition_result r)
112 {
113     switch (r) {
114     case OSTR_SUCCESS:
115         return "success";
116     case OSTR_COURIER_IS_NOT_SET_YET:
117         return "courier is not set yet";
118     case OSTR_INVALID_NEW_STATUS_FOR_THIS_KIND_OF_ORDER:
119         return "invalid new status for this kind of order";
120     }
121     assert(0 && "unreachable");
122 }
123
124 enum order_status_transition_result order_change_status(struct order *o, enum order_status
    new_status)
125 {
126     enum order_status_transition_result ret;
127     ret = o->kind->vptr->allows_transition(o->kind, o, new_status);
128     if (ret != OSTR_SUCCESS) return ret;
129     o->status = new_status;
130     return OSTR_SUCCESS;
131 }
132
133 size_t order_get_items_count(const struct order *o)
134 {
135     return o->info.items_count;
136 }
137
138 int order_get_id(struct order const *o)
139 {
140     return o->info.id;
141 }
142
143 enum order_status order_get_status(const struct order *order)
144 {
145     return order->status;
146 }
147
148 courier *order_get_courier(const struct order *order)
149 {
150     return order->courier;
151 }
152
153 double order_calc_cost(const struct order *o)

```

```

154 {
155     return (double)o->kind->vptr->calc_cost(o->kind, o);
156 }
157
158 double order_calc_eta_hours(const struct order *o)
159 {
160     return (double)o->kind->vptr->calc_estimated_time(o->kind, o);
161 }
162
163 int order_can_change_status(const struct order *o, enum order_status to)
164 {
165     return o->kind->vptr->allows_transition(o->kind, o, to) == OSTR_SUCCESS;
166 }
167
168 int order_can_change_kind(const struct order *o)
169 {
170     return o->status == ORDER_STATUS_PACKING;
171 }
172
173 void order_set_order_kind(struct order *o, order_kind *kind)
174 {
175     o->kind = kind;
176     if (o->courier && !kind->vptr->needs_courier(kind, o))
177         o->courier = NULL;
178 }

```

src/service/destination_address.c:

```

1 #include "destination_address.h"
2
3 #include <stdio.h>
4
5 destination_address *destination_address_create(struct context *ctx, const char *city,
6     const char *street, const char *building, const char *apartment, const char *comment)
7 {
8     struct destination_address *dest = context_alloc(ctx, sizeof *dest);
9     dest->city = city;
10    dest->street = street;
11    dest->building = building;
12    dest->apartment = apartment;
13    dest->comment = comment;
14    return dest;
15 }
16
17 void destination_address_print(destination_address *addr, int pad)
18 {
19     printf("%*sCity: %s\n", pad, "", addr->city);
20     printf("%*sStreet: %s\n", pad, "", addr->street);
21     printf("%*sBuilding: %s\n", pad, "", addr->building);
22     if (addr->apartment) printf("%*sApartment: %s\n", pad, "", addr->apartment);
23     if (addr->comment) printf("%*sComment: \n%s%s", pad, "", pad+2, "", addr->comment);

```

src/service/client.c:

```

1 #include "client.h"
2
3 #include <stdio.h>

```

```

4 | #include <string.h>
5 |
6 | client_data *client_heapify(struct context *ctx, client_data *cdata)
7 | {
8 |     struct client_data *data = context_alloc(ctx, sizeof *data);
9 |     memcpy(data, cdata, sizeof *data);
10 |    return data;
11 | }
12 |
13 | void client_print(client_data *client, int pad)
14 | {
15 |     printf("%sName: %s\n", pad, "", client->name);
16 |     printf("%sPhone: %s\n", pad, "", client->phone);
17 |     printf("%sEmail: %s\n", pad, "", client->email);
18 | }
19 |
20 | unsigned client_hasheq(enum hasheq_op op, const void *pa, const void *pb)
21 | {
22 |     client_data *const *a, *const *b;
23 |     a = pa;
24 |     b = pb;
25 |     switch (op) {
26 |     case HASHEQ_EQ:
27 |         return strcmp((*a)->name, (*b)->name) == 0;
28 |     case HASHEQ_HASH:
29 |         return fnv1((*a)->name, strlen((*a)->name));
30 |     }
31 |     return -1;
32 | }
33 |
34 | int client_validate_phone(const char *phone)
35 | {
36 |     size_t n = 0, digits = 0;
37 |     if (!phone) return 0;
38 |     for (; *phone; ++phone, ++n) {
39 |         if (*phone >= '0' && *phone <= '9') { ++digits; continue; }
40 |         if (strchr("+ - ()", *phone)) continue;
41 |         return 0;
42 |     }
43 |     return n > 0 && digits >= 5;
44 | }
45 |
46 | int client_validate_email(const char *email)
47 | {
48 |     const char *at, *dot;
49 |     if (!email || !*email) return 0;
50 |     at = strchr(email, '@');
51 |     if (!at || at == email) return 0;
52 |     if (strchr(at + 1, '@')) return 0;
53 |     dot = strrchr(at + 1, '.');
54 |     if (!dot || dot == at + 1 || !dot[1]) return 0;
55 |     return 1;
56 | }

```

src/tui/errors.c:

```

1 | #include "errors.h"
2 |

```

```

3 | #include <stdio.h>
4 |
5 | void print_order_service_error(enum order_service_error err)
6 | {
7 |     fprintf(stderr, "error: service errored: ");
8 |     switch (err) {
9 |     case OSE_NONE:
10 |         fprintf(stderr, "no error\n");
11 |         break;
12 |     case OSE_CLIENT_DOESNOT_EXISTS:
13 |         fprintf(stderr, "user does not exists\n");
14 |         break;
15 |     case OSE_NO_SUCH_ORDER_WITH_ID:
16 |         fprintf(stderr, "no such order with the id\n");
17 |         break;
18 |     case OSE_COURIER_IS_NOT_SET_YET:
19 |         fprintf(stderr, "courier is not set yet\n");
20 |         break;
21 |     case OSE_INVALID_NEW_STATUS:
22 |         fprintf(stderr, "invalid new status\n");
23 |         break;
24 |     case OSE_KIND_DOES_NOT_NEED_COURIER:
25 |         fprintf(stderr, "this order kind does not need a courier\n");
26 |         break;
27 |     case OSE_KIND_FROZEN:
28 |         fprintf(stderr, "order kind is frozen after packing started\n");
29 |         break;
30 |     }
31 | }

```

src/tui/menu.c:

```

1 | #include "menu.h"
2 |
3 | #include <stdio.h>
4 | #include <stdlib.h>
5 | #include <readline/readline.h>
6 |
7 | struct menu_state_list {
8 |     struct menu_state *state;
9 |     struct menu_state_list *next;
10 | };
11 |
12 | struct menu {
13 |     struct menu_state_list *stack;
14 |     void *userdata;
15 |     char *input;
16 |     int match;
17 | };
18 |
19 | static void menu__help(struct menu *m);
20 | static void menu__quit(struct menu *m);
21 |
22 | static const struct menu_command common_commands[] = {
23 |     {"help", "prints this help message", menu__help},
24 |     {"quit", "terminates program", menu__quit},
25 |     {"exit", "", menu__quit},
26 |     {NULL, NULL, NULL}

```

```

27 };
28
29 struct menu *menu_create(struct context *c)
30 {
31     struct menu *m;
32     m = context_alloc(c, sizeof *m);
33     return m;
34 }
35
36 static void menu__on_unmatched_command(struct menu *m)
37 {
38     if (m->match) return;
39     printf("Type \"help\" for getting all commands for current menu\n");
40 }
41
42 static int menu__match_command(struct menu *m, struct menu_command const *cmd)
43 {
44     for (; cmd->command; cmd++) {
45         if (strcmp(m->input, cmd->command) != 0) continue;
46         cmd->run(m);
47         m->match = 1;
48         return 1;
49     }
50     return 0;
51 }
52
53 static void menu__dispatch(struct menu *m)
54 {
55     m->match = 0;
56     if (menu__match_command(m, m->stack->state->commands)) return;
57     if (menu__match_command(m, common_commands)) return;
58 }
59
60 static void menu__collect_input(struct menu *m, const char *prompt)
61 {
62     free(m->input);
63     m->input = readline(prompt);
64     if (!m->input) menu__quit(m);
65 }
66
67 const char *menu_prompt(struct menu *m, const char *prompt)
68 {
69     menu__collect_input(m, prompt);
70     return m->input;
71 }
72
73 void menu_input(struct menu *m)
74 {
75     menu__on_unmatched_command(m);
76     menu__collect_input(m, "> ");
77     menu__dispatch(m);
78 }
79
80 static void menu__quit(struct menu *m)
81 {
82     free(m->input);
83     context_quit(context_from_alloc(m));
84 }

```

```

85
86 static void menu__help(struct menu *m)
87 {
88     const struct menu_command *p[3], **i;
89     int l, max_len = 0;
90     p[0] = m->stack->state->commands;
91     p[1] = common_commands;
92     p[2] = NULL;
93
94     for (i = p; *i; ++i) {
95         for (; (*i)->command; ++(*i)) {
96             l = strlen((*i)->command);
97             max_len = max_len < l ? l : max_len;
98         }
99     }
100
101     p[0] = m->stack->state->commands;
102     p[1] = common_commands;
103     p[2] = NULL;
104
105     for (i = p; *i; ++i) {
106         for (; (*i)->command; ++(*i)) {
107             printf("    %s %s\n", -max_len, (*i)->command, (*i)->description);
108         }
109     }
110 }
111
112 void menu_push_state(struct menu *m, struct menu_state *state)
113 {
114     struct context *ctx = context_from_alloc(m);
115     struct menu_state_list *node;
116
117     node = context_alloc(ctx, sizeof *node);
118     node->state = state;
119     node->next = m->stack;
120     m->stack = node;
121 }
122
123 void *menu_get_userdata(struct menu *m)
124 {
125     return m->userdata;
126 }
127
128 const char *menu_get_input(struct menu *m)
129 {
130     return m->input;
131 }
132
133 void menu_set_userdata(struct menu *m, void *userdata)
134 {
135     m->userdata = userdata;
136 }
137
138 struct menu_state *menu_get_state(struct menu *m)
139 {
140     return m->stack->state;
141 }
142

```

```

143 void menu_pop_state(struct menu *m)
144 {
145     struct menu_state_list *next = m->stack->next;
146     context_free(m->stack);
147     m->stack = next;
148 }

```

src/tui/menus/yes_no_menu.c:

```

1 #include "../menu.h"
2
3 struct menu_yes_no {
4     struct menu_state base;
5     menu_action on_yes;
6     menu_action on_no;
7     void *userdata;
8 };
9
10 static void yes(struct menu *);
11 static void no(struct menu *);
12
13 static const struct menu_command commands[] = {
14     {"y",          "yes answer", yes},
15     {"yes",        "", yes},
16     {"да",         "", yes},
17     {"ok",         "", yes},
18     {"of course",  "", yes},
19     {"YEAH",       "", yes},
20     {"n",          "no answer", no},
21     {"нет",        "", no},
22     {"no",         "", no},
23     {"NO",         "", no},
24     {"not ok",     "", no},
25     {NULL, NULL, NULL}
26 };
27
28 struct menu_state *yes_no_menu_create(struct context *ctx,
29                                     menu_action on_yes,
30                                     menu_action on_no,
31                                     void *userdata)
32 {
33     struct menu_yes_no *state = context_alloc(ctx, sizeof *state);
34     state->base.commands = commands;
35     state->on_yes = on_yes;
36     state->on_no = on_no;
37     state->userdata = userdata;
38     return &state->base;
39 }
40
41 static void yes(struct menu *m)
42 {
43     struct menu_yes_no *state = (void *)menu_get_state(m);
44     menu_pop_state(m);
45     if (state->on_yes) state->on_yes(m, state->userdata);
46     context_free(state);
47 }
48
49 static void no(struct menu *m)

```

```

50 {
51     struct menu_yes_no *state = (void *)menu_get_state(m);
52     menu_pop_state(m);
53     if (state->on_no) state->on_no(m, state->userdata);
54     context_free(state);
55 }

```

src/tui/menus/main_menu.c:

```

1  #include "../menus.h"
2  #include "../errors.h"
3  #include "../../service/order_service.h"
4
5  #include <stdio.h>
6  #include <stdlib.h>
7
8  static void new_order(struct menu *);
9  static void list_orders(struct menu *);
10 static void add_items(struct menu *);
11 static void manage(struct menu *);
12 static void assign(struct menu *);
13 static void status(struct menu *);
14
15 static const struct menu_command commands[] = {
16     {"new_order",    "create a new order",          new_order},
17     {"list_orders", "list the existing orders",      list_orders},
18     {"items",        "add items to an order",        add_items},
19     {"manage",       "choose delivery kind, show cost and ETA", manage},
20     {"assign",       "assign courier to the order",   assign},
21     {"status",       "change order status and show current info", status},
22     {NULL, NULL, NULL}
23 };
24
25 struct menu_state *main_menu_create(struct context *ctx)
26 {
27     struct menu_state *s;
28     s = context_alloc(ctx, sizeof *s);
29     s->commands = commands;
30     return s;
31 }
32
33 static int prompt_order_id_with_checking_the_db___wasteful_but_otherwise_ux_is_shit(struct
    menu *m, struct order **order)
34 {
35     struct order *o;
36     struct order_service *s;
37     int id;
38
39     s = menu_get_userdata(m);
40     id = atoi(menu_prompt(m, "order id: "));
41     o = order_service_find(s, id);
42     if (!o) {
43         print_order_service_error(order_service_get_error(s));
44         if (order) *order = NULL;
45         return -1;
46     }
47     if (order) *order = o;
48     return id;

```

```

49 }
50
51 static void new_order(struct menu *m)
52 {
53     struct context *ctx = context_from_alloc(m);
54     menu_push_state(m, new_order_menu_create(ctx));
55 }
56
57 static void list_orders(struct menu *m)
58 {
59     struct order_service *s;
60     s = menu_get_userdata(m);
61     order_service_list(s);
62 }
63
64 static void add_items(struct menu *m)
65 {
66     struct context *ctx;
67     int id;
68
69     ctx = context_from_alloc(m);
70     id = prompt_order_id_with_checking_the_db___wasteful_but_otherwise_ux_is_shit(m, NULL);
71     if (id < 0) return;
72     menu_push_state(m, items_menu_create(ctx, id));
73 }
74
75 static void manage(struct menu *m)
76 {
77     struct context *ctx;
78     struct order *o;
79     int id;
80
81     ctx = context_from_alloc(m);
82
83     id = prompt_order_id_with_checking_the_db___wasteful_but_otherwise_ux_is_shit(m, &o);
84     if (id < 0) return;
85     if (!order_can_change_kind(o)) {
86         fprintf(stderr, "error: order kind is frozen after packing started\n");
87         return;
88     }
89     menu_push_state(m, manage_menu_create(ctx, id));
90 }
91
92 static void status(struct menu *m)
93 {
94     struct context *ctx;
95     struct order *o;
96     int id;
97
98     ctx = context_from_alloc(m);
99     id = prompt_order_id_with_checking_the_db___wasteful_but_otherwise_ux_is_shit(m, &o);
100    if (id < 0) return;
101    order_print(o);
102    menu_push_state(m, status_menu_create(ctx, id, o));
103 }
104
105 struct courier_stash {
106     int id;

```

```

107     char *name;
108 };
109
110 static void courier_assign_go(struct menu *m, void *userdata, int has_car)
111 {
112     struct courier_stash *st = userdata;
113     struct order_service *svc = menu_get_userdata(m);
114     int notes = has_car ? COURIER_NOTES_HAS_CAR : COURIER_NOTES_NONE;
115
116     order_service_assign_courier(svc, st->id, st->name, notes);
117     if (order_service_get_error(svc) != OSE_NONE)
118         print_order_service_error(order_service_get_error(svc));
119     context_free(st);
120 }
121
122 static void courier_assign_yes(struct menu *m, void *userdata)
123 {
124     courier_assign_go(m, userdata, 1);
125 }
126
127 static void courier_assign_no(struct menu *m, void *userdata)
128 {
129     courier_assign_go(m, userdata, 0);
130 }
131
132 static void assign(struct menu *m)
133 {
134     struct context *ctx;
135     struct courier_stash *st;
136     int id;
137
138     ctx = context_from_alloc(m);
139
140     id = prompt_order_id_with_checking_the_db___wasteful_but_otherwise_ux_is_shit(m, NULL);
141     if (id < 0) return;
142
143     st = context_alloc(ctx, sizeof *st);
144     st->id = id;
145     st->name = context_strdup(ctx, menu_prompt(m, "courier name: "));
146
147     printf("Does the courier have a car?\n");
148     menu_push_state(m, yes_no_menu_create(ctx, courier_assign_yes, courier_assign_no, st));
149 }

```

src/tui/menus/new_order_menu.c:

```

1 #include "../menus.h"
2 #include "../errors.h"
3 #include "../../service/order_service.h"
4
5 #include <stdio.h>
6 #include <string.h>
7
8 struct order_draft {
9     char *client_name;
10    char *city;
11    char *street;
12    char *building;

```

```

13     char *apartment;
14     char *comment;
15 };
16
17 struct menu_new_order {
18     struct menu_state base;
19     struct order_draft draft;
20 };
21
22 static void set_client(struct menu *);
23 static void set_address(struct menu *);
24 static void show(struct menu *);
25 static void create(struct menu *);
26 static void cancel(struct menu *);
27
28 static const struct menu_command commands[] = {
29     {"client", "set client name",      set_client},
30     {"address", "set delivery address", set_address},
31     {"show",    "show current draft",   show},
32     {"create",  "create the order",     create},
33     {"cancel",  "cancel and discard draft", cancel},
34     {NULL, NULL, NULL}
35 };
36
37 struct menu_state *new_order_menu_create(struct context *ctx)
38 {
39     struct menu_new_order *s = context_alloc(ctx, sizeof *s);
40     s->base.commands = commands;
41     /* draft обнулён context_allocом- */
42     return &s->base;
43 }
44
45 static char *dup_or_null(struct context *ctx, const char *v)
46 {
47     if (!v || !*v) return NULL;
48     return context_strdup(ctx, v);
49 }
50
51 static void set_client(struct menu *m)
52 {
53     struct menu_new_order *s = (void *)menu_get_state(m);
54     struct context *ctx = context_from_alloc(m);
55     const char *v = menu_prompt(m, "client name: ");
56     context_free(s->draft.client_name);
57     s->draft.client_name = dup_or_null(ctx, v);
58 }
59
60 static void set_address(struct menu *m)
61 {
62     struct menu_new_order *s = (void *)menu_get_state(m);
63     struct context *ctx = context_from_alloc(m);
64     const char *v;
65
66     v = menu_prompt(m, "city: ");
67     context_free(s->draft.city);
68     s->draft.city = dup_or_null(ctx, v);
69
70     v = menu_prompt(m, "street: ");

```

```

71 context_free(s->draft.street);
72 s->draft.street = dup_or_null(ctx, v);
73
74 v = menu_prompt(m, "building: ");
75 context_free(s->draft.building);
76 s->draft.building = dup_or_null(ctx, v);
77
78 v = menu_prompt(m, "apartment (empty to skip): ");
79 context_free(s->draft.apartment);
80 s->draft.apartment = dup_or_null(ctx, v);
81
82 v = menu_prompt(m, "comment (empty to skip): ");
83 context_free(s->draft.comment);
84 s->draft.comment = dup_or_null(ctx, v);
85 }
86
87 static void show(struct menu *m)
88 {
89     struct menu_new_order *s = (void *)menu_get_state(m);
90     struct order_draft *d = &s->draft;
91
92     printf("Draft order:\n");
93     printf("  client:   %s\n", d->client_name ? d->client_name : "(unset)");
94     printf("  city:     %s\n", d->city ? d->city : "(unset)");
95     printf("  street:   %s\n", d->street ? d->street : "(unset)");
96     printf("  building: %s\n", d->building ? d->building : "(unset)");
97     printf("  apartment: %s\n", d->apartment ? d->apartment : "(unset)");
98     printf("  comment:  %s\n", d->comment ? d->comment : "(unset)");
99 }
100
101 static void draft_clear(struct order_draft *d)
102 {
103     context_free(d->client_name);
104     context_free(d->city);
105     context_free(d->street);
106     context_free(d->building);
107     context_free(d->apartment);
108     context_free(d->comment);
109     memset(d, 0, sizeof *d);
110 }
111
112 static int draft_is_complete(const struct order_draft *d)
113 {
114     return d->client_name && *d->client_name
115         && d->city && *d->city
116         && d->street && *d->street
117         && d->building && *d->building;
118 }
119
120 static void cancel(struct menu *m)
121 {
122     struct menu_new_order *s = (void *)menu_get_state(m);
123     draft_clear(&s->draft);
124     menu_pop_state(m);
125     context_free(s);
126 }
127
128 /* Колбэки для yes_no_menu. На момент вызова yes_no_menu уже снят,

```

```

129  мы на верхушке стека - это new_order_menu. */
130 static void on_create_client(struct menu *m, void *userdata)
131 {
132     struct menu_new_order *s = (void *)menu_get_state(m);
133     struct context *ctx = context_from_alloc(m);
134     struct client_data prefill;
135     (void)userdata;
136
137     prefill.name = s->draft.client_name;
138     prefill.email = NULL;
139     prefill.phone = NULL;
140
141     menu_push_state(m, client_creation_menu_create(ctx, &prefill));
142 }
143
144 static void on_skip_client(struct menu *m, void *userdata)
145 {
146     (void)m;
147     (void)userdata;
148     /* просто остаёмся в new_order_menu; юзер может поправить поле
149        "client" или снова нажать "create" */
150 }
151
152 static void create(struct menu *m)
153 {
154     struct context *ctx = context_from_alloc(m);
155     struct menu_new_order *s = (void *)menu_get_state(m);
156     struct order_service *svc = menu_get_userdata(m);
157     struct order *o;
158     destination_address *addr;
159     enum order_service_error err;
160
161     if (!draft_is_complete(&s->draft)) {
162         fprintf(stderr, "error: fields 'client', 'city', 'street', "
163                     "'building' are required\n");
164         return;
165     }
166
167     addr = destination_address_create(ctx,
168                                     s->draft.city,
169                                     s->draft.street,
170                                     s->draft.building,
171                                     s->draft.apartment,
172                                     s->draft.comment);
173
174     o = order_service_create_order(svc, s->draft.client_name, addr);
175     if (o) {
176         order_print(o);
177         /* draft строки - остаются в арене и живут столько же, сколько
178            адрес заказа - то есть до выхода. Не чистим, иначе addr
179            в ordere' будет висеть на освобождённой памяти. */
180         menu_pop_state(m);
181         context_free(s);
182         return;
183     }
184
185     err = order_service_get_error(svc);
186     if (err != OSE_CLIENT_DOESNOT_EXISTS) {

```

```

187     print_order_service_error(err);
188     return;
189 }
190
191 printf("Client '%s' not found. Create new client?\n", s->draft.client_name);
192 menu_push_state(m, yes_no_menu_create(ctx, on_create_client, on_skip_client, NULL));
193 }

```

src/tui/menus/manage_menu.c:

```

1 #include "../menus.h"
2 #include "../errors.h"
3 #include "../../service/order_service.h"
4
5 #include <stdio.h>
6
7 struct menu_manage {
8     struct menu_state base;
9     int order_id;
10 };
11
12 static void pick_standard(struct menu *);
13 static void pick_express(struct menu *);
14 static void pick_pickup(struct menu *);
15
16 static const struct menu_command commands[] = {
17     {"standard", "standard delivery", pick_standard},
18     {"express", "express delivery", pick_express},
19     {"pickup", "self pickup", pick_pickup},
20     {NULL, NULL, NULL}
21 };
22
23 static void finish(struct menu *m, order_kind *k)
24 {
25     struct menu_manage *s;
26     struct order_service *svc;
27     struct order *o;
28
29     s = (struct menu_manage *)menu_get_state(m);
30     svc = menu_get_userdata(m);
31
32     order_service_set_kind(svc, s->order_id, k);
33     if (order_service_get_error(svc) != OSE_NONE) {
34         print_order_service_error(order_service_get_error(svc));
35         menu_pop_state(m);
36         context_free(s);
37         return;
38     }
39
40     o = order_service_find(svc, s->order_id);
41     if (o) {
42         printf("Delivery: %s\n", k->name);
43         printf("Cost: %.2f\n", order_calc_cost(o));
44         printf("ETA: %.0f h\n", order_calc_eta_hours(o));
45     }
46
47     menu_pop_state(m);
48     context_free(s);

```

```

49 }
50
51 static void pick_standard(struct menu *m)
52 {
53     struct order_service *svc;
54     svc = menu_get_userdata(m);
55     finish(m, order_service_get_standard_kind(svc));
56 }
57
58 static void pick_express(struct menu *m)
59 {
60     struct order_service *svc;
61     svc = menu_get_userdata(m);
62     finish(m, order_service_get_express_kind(svc));
63 }
64
65 static void pick_pickup(struct menu *m)
66 {
67     struct order_service *svc;
68     svc = menu_get_userdata(m);
69     finish(m, order_service_get_pickup_kind(svc));
70 }
71
72 struct menu_state *manage_menu_create(struct context *ctx, int order_id)
73 {
74     struct menu_manage *s;
75     s = context_alloc(ctx, sizeof *s);
76     s->base.commands = commands;
77     s->order_id = order_id;
78     return &s->base;
79 }

```

src/tui/menus/client_creation_menu.c:

```

1 #include "../menus.h"
2 #include "../../service/client.h"
3 #include "../../service/order_service.h"
4
5 #include <stdio.h>
6
7 struct menu_client_creation {
8     struct menu_state base;
9     struct client_data *data;
10 };
11
12 static void show(struct menu *);
13 static void name(struct menu *);
14 static void email(struct menu *);
15 static void phone(struct menu *);
16 static void done(struct menu *);
17 static void cancel(struct menu *);
18
19 static const struct menu_command commands[] = {
20     {"show", "show constructed client", show},
21     {"name", "set client name", name},
22     {"email", "set client email", email},
23     {"phone", "set client phone", phone},
24     {"done", "finish client creation", done},

```

```

25     {"cancel", "abort and return",      cancel},
26     {NULL, NULL, NULL}
27 };
28
29 static char *opt_dup(struct context *ctx, const char *v)
30 {
31     if (!v || !*v) return NULL;
32     return context_strdup(ctx, v);
33 }
34
35 struct menu_state *client_creation_menu_create(struct context *ctx, const struct
    client_data *initial)
36 {
37     struct menu_client_creation *state = context_alloc(ctx, sizeof *state);
38     state->base.commands = commands;
39     state->data = context_alloc(ctx, sizeof *state->data);
40
41     if (initial) {
42         state->data->name = opt_dup(ctx, initial->name);
43         state->data->email = opt_dup(ctx, initial->email);
44         state->data->phone = opt_dup(ctx, initial->phone);
45     }
46
47     return &state->base;
48 }
49
50 static void show(struct menu *m)
51 {
52     struct menu_client_creation *state = (void *)menu_get_state(m);
53
54     printf("Client:\n");
55     printf("  name:  %s\n", state->data->name ? state->data->name : "(unset)");
56     printf("  email: %s\n", state->data->email ? state->data->email : "(unset)");
57     printf("  phone: %s\n", state->data->phone ? state->data->phone : "(unset)");
58 }
59
60 static void name(struct menu *m)
61 {
62     struct menu_client_creation *state = (void *)menu_get_state(m);
63     const char *v = menu_prompt(m, "name: ");
64     context_free((void *)state->data->name);
65     state->data->name = opt_dup(context_from_alloc(m), v);
66 }
67
68 static void email(struct menu *m)
69 {
70     struct menu_client_creation *state = (void *)menu_get_state(m);
71     const char *v = menu_prompt(m, "email: ");
72     context_free((void *)state->data->email);
73     state->data->email = opt_dup(context_from_alloc(m), v);
74 }
75
76 static void phone(struct menu *m)
77 {
78     struct menu_client_creation *state = (void *)menu_get_state(m);
79     const char *v = menu_prompt(m, "phone: ");
80     context_free((void *)state->data->phone);
81     state->data->phone = opt_dup(context_from_alloc(m), v);

```

```

82 }
83
84 static void done(struct menu *m)
85 {
86     struct menu_client_creation *state = (void *)menu_get_state(m);
87     struct order_service *svc = menu_get_userdata(m);
88     struct client_data *cd = state->data;
89
90     if (!cd->name || !*cd->name) {
91         fprintf(stderr, "error: name is required\n");
92         return;
93     }
94     if (!client_validate_email(cd->email)) {
95         fprintf(stderr, "error: invalid email (expected name@domain.tld)\n");
96         return;
97     }
98     if (!client_validate_phone(cd->phone)) {
99         fprintf(stderr, "error: invalid phone (at least 5 digits, allowed: digits, + -
100         space ( ))\n");
101         return;
102     }
103     if (order_service_register_client(svc, cd)) {
104         fprintf(stderr, "error: client '%s' already exists; pick a different name\n",
105         cd->name);
106         return;
107     }
108
109     printf("Client '%s' registered. Type 'create' to finish the order, or 'quit' to
110     exit.\n", cd->name);
111
112     menu_pop_state(m);
113     context_free(state);
114 }
115
116 static void cancel(struct menu *m)
117 {
118     struct menu_client_creation *state = (void *)menu_get_state(m);
119     /* state->data и его строки остаются в арене до выхода из контекста;
120     освободить их не критично, но чтобы не плодить мусор - чистим. */
121     context_free((void *)state->data->name);
122     context_free((void *)state->data->email);
123     context_free((void *)state->data->phone);
124     context_free(state->data);
125     menu_pop_state(m);
126     context_free(state);
127 }

```

src/tui/menus/items_menu.c:

```

1 #include "../menus.h"
2 #include "../../service/order_service.h"
3
4 #include <stdio.h>
5 #include <stdlib.h>
6
7 struct menu_items {
8     struct menu_state base;
9     int order_id;

```

```

10};
11
12static void add(struct menu *);
13static void done(struct menu *);
14
15static const struct menu_command commands[] = {
16    {"add", "add a new item to the order", add},
17    {"done", "finish adding items", done},
18    {NULL, NULL, NULL}
19};
20
21struct menu_state *items_menu_create(struct context *ctx, int order_id)
22{
23    struct menu_items *s = context_alloc(ctx, sizeof *s);
24    s->base.commands = commands;
25    s->order_id = order_id;
26    return &s->base;
27}
28
29static void add(struct menu *m)
30{
31    struct menu_items *s = (void *)menu_get_state(m);
32    struct order_service *svc = menu_get_userdata(m);
33    const char *qty_s, *price_s;
34    char *name;
35    int qty;
36    float price;
37
38    /* menu_prompt reuses m->input across calls; copy the name */
39    name = context_strdup(context_from_alloc(m), menu_prompt(m, "item name: "));
40
41    qty_s = menu_prompt(m, "quantity: ");
42    qty = atoi(qty_s);
43    if (qty <= 0) {
44        fprintf(stderr, "error: quantity must be positive\n");
45        return;
46    }
47
48    price_s = menu_prompt(m, "price: ");
49    price = (float)atof(price_s);
50    if (price < 0.0f) {
51        fprintf(stderr, "error: price must be non-negative\n");
52        return;
53    }
54
55    order_service_add_item(svc, s->order_id, name, qty, price);
56    printf("Item added.\n");
57}
58
59static void done(struct menu *m)
60{
61    struct menu_items *s = (void *)menu_get_state(m);
62    struct order_service *svc = menu_get_userdata(m);
63    struct order *o = order_service_find(svc, s->order_id);
64
65    if (o) order_print(o);
66
67    menu_pop_state(m);

```

```

68     context_free(s);
69 }

```

src/tui/menus/status_menu.c:

```

1  #include "../menus.h"
2  #include "../errors.h"
3  #include "../../service/order_service.h"
4
5  #include <stdio.h>
6  #include <string.h>
7
8  static const struct status_entry {
9      const char *cmd;
10     const char *desc;
11     enum order_status status;
12 } status_table[] = {
13     {"packing",          "back to packing",  ORDER_STATUS_PACKING},
14     {"ready_for_pickup", "ready for pickup",  ORDER_STATUS_READY_FOR_PICKUP},
15     {"on_the_way",       "start delivery",   ORDER_STATUS_ON_THE_WAY},
16     {"delivered",        "mark delivered",   ORDER_STATUS_DELIVERED},
17     {"picked",           "mark picked up",   ORDER_STATUS_PICKED}
18 };
19
20 #define STATUS_TABLE_N (sizeof status_table / sizeof status_table[0])
21
22 struct menu_status {
23     struct menu_state base;
24     int order_id;
25     struct menu_command cmds[STATUS_TABLE_N + 2];
26 };
27
28 static void change_status(struct menu *m)
29 {
30     struct menu_status *s;
31     struct order_service *svc;
32     struct order *o;
33     const char *input;
34     size_t i;
35
36     s = (struct menu_status *)menu_get_state(m);
37     svc = menu_get_userdata(m);
38     input = menu_get_input(m);
39
40     for (i = 0; i < STATUS_TABLE_N; ++i) {
41         if (strcmp(input, status_table[i].cmd) == 0) break;
42     }
43     if (i == STATUS_TABLE_N) return;
44
45     order_service_change_status(svc, s->order_id, status_table[i].status);
46     if (order_service_get_error(svc) != OSE_NONE)
47         print_order_service_error(order_service_get_error(svc));
48
49     o = order_service_find(svc, s->order_id);
50     if (o) order_print(o);
51
52     menu_pop_state(m);
53     context_free(s);

```

```

54 }
55
56 static void cancel(struct menu *m)
57 {
58     struct menu_status *s = (struct menu_status *)menu_get_state(m);
59     menu_pop_state(m);
60     context_free(s);
61 }
62
63 struct menu_state *status_menu_create(struct context *ctx, int order_id, struct order *o)
64 {
65     struct menu_status *s;
66     size_t i, n;
67
68     s = context_alloc(ctx, sizeof *s);
69
70     n = 0;
71     for (i = 0; i < STATUS_TABLE_N; ++i) {
72         if (!order_can_change_status(o, status_table[i].status)) continue;
73         s->cmds[n].command = status_table[i].cmd;
74         s->cmds[n].description = status_table[i].desc;
75         s->cmds[n].run = change_status;
76         ++n;
77     }
78     s->cmds[n].command = "cancel";
79     s->cmds[n].description = "cancel and return";
80     s->cmds[n].run = cancel;
81     ++n;
82     s->cmds[n].command = NULL;
83     s->cmds[n].description = NULL;
84     s->cmds[n].run = NULL;
85
86     s->base.commands = s->cmds;
87     s->order_id = order_id;
88     return &s->base;
89 }

```

src/runtime/context.c:

```

1 #include "context.h"
2
3 #include <assert.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <setjmp.h>
7 #include <string.h>
8
9 #define MAGIC 0xc7e3ede
10
11 struct context_alloc_hdr {
12     int magic;
13     struct context_alloc_hdr *next, *prev;
14     struct context *parent;
15 };
16
17 struct context {
18     struct context_alloc_hdr *allocations;
19     jmp_buf exception_handler;

```

```

20 | };
21 |
22 | void *context_alloc(struct context *c, size_t size)
23 | {
24 |     struct context_alloc_hdr *hdr;
25 |     hdr = malloc(sizeof *hdr + size);
26 |     if (!hdr) longjmp(c->exception_handler, CTX_E_MALLOC_FAILURE);
27 |
28 |     memset(hdr, 0, sizeof *hdr + size);
29 |     hdr->magic = MAGIC;
30 |     hdr->parent = c;
31 |     if (c->allocations)
32 |         c->allocations->prev = hdr;
33 |     hdr->next = c->allocations;
34 |     c->allocations = hdr;
35 |
36 |     return hdr + 1;
37 | }
38 |
39 | void context_quit(struct context *c)
40 | {
41 |     longjmp(c->exception_handler, CTX_E_QUIT);
42 | }
43 |
44 | void context_enter(void (*f)(struct context *))
45 | {
46 |     struct context *c;
47 |     enum context_exception e;
48 |     c = malloc(sizeof *c);
49 |     memset(c, 0, sizeof *c);
50 |     if (!c) {
51 |         fprintf(stderr, "ERROR: could not enter context\n");
52 |         exit(1);
53 |     }
54 |     e = setjmp(c->exception_handler);
55 |     switch (e) {
56 |     case CTX_E_NONE: break;
57 |     case CTX_E_QUIT: goto quit;
58 |     case CTX_E_MALLOC_FAILURE:
59 |         fprintf(stderr, "ERROR: malloc failed\n");
60 |         exit(1);
61 |         break;
62 |     }
63 |
64 |     f(c);
65 |
66 | quit:
67 |     while (c->allocations) {
68 |         void *next = c->allocations->next;
69 |         free(c->allocations);
70 |         c->allocations = next;
71 |     }
72 |     free(c);
73 | }
74 |
75 | static struct context_alloc_hdr *context__get_alloc_header(void *p)
76 | {
77 |     struct context_alloc_hdr *h = p;

```

```

78     assert(h[-1].magic == MAGIC);
79     return h - 1;
80 }
81
82 struct context *context_from_alloc(void *p)
83 {
84     return context__get_alloc_header(p)->parent;
85 }
86
87 void context_free(void *p)
88 {
89     struct context_alloc_hdr *h;
90     struct context *c;
91
92     if (!p) return;
93
94     h = context__get_alloc_header(p);
95     c = h->parent;
96
97     if (h->next) h->next->prev = h->prev;
98     if (h->prev) h->prev->next = h->next;
99     if (c->allocations == h) c->allocations = h->next;
100
101     free(h);
102 }
103
104 char *context_strdup(struct context *c, const char *cstr)
105 {
106     char *copy = context_alloc(c, strlen(cstr) + 1);
107     strcpy(copy, cstr);
108     return copy;
109 }

```

src/hashmap.c:

```

1  #include "hashmap.h"
2
3  #include <string.h>
4  #include <stdlib.h>
5
6  struct hashmap {
7      void *items;
8      unsigned char *bitmap;
9      size_t count;
10     size_t cap;
11     size_t itemsize;
12     hasheq_f hasheq;
13 };
14
15 struct hashmap *hashmap_create(struct context *ctx, hasheq_f heq, size_t itemsize)
16 {
17     struct hashmap *hm = context_alloc(ctx, sizeof *hm);
18     hm->itemsize = itemsize;
19     hm->hasheq = heq;
20     return hm;
21 }
22
23 enum {

```

```

24     ICTL_IS_SET,
25     ICTL_SET
26 };
27
28 static int hashmap_index_ctl(struct hashmap *base, unsigned int index, int ctl)
29 {
30     switch (ctl) {
31     case ICTL_IS_SET:
32         return !(base->bitmap[index>>3] & (1<<(index&7)));
33     case ICTL_SET:
34         base->bitmap[index>>3] |= 1<<(index&7);
35         return 0;
36     default: abort();
37     }
38 }
39
40 static void *hashmap_slot_at(struct hashmap *base, unsigned index)
41 {
42     return (char *)base->items + index*base->itemsize;
43 }
44
45 static unsigned int hashmap_keyindex(struct hashmap *base, const void *key, int
    find_insert_slot)
46 {
47     unsigned int keyhash = base->hasheq(HASHEQ_HASH, key, NULL);
48     size_t limit = 3*base->cap/4;
49     size_t i;
50
51     for (i = 0; i < limit; ++i) {
52         int eq = 0, isset;
53         size_t index = (keyhash+i)%base->cap;
54         isset = hashmap_index_ctl(base, index, ICTL_IS_SET);
55         if (isset) eq = base->hasheq(HASHEQ_EQ, key, hashmap_slot_at(base, index));
56         if (find_insert_slot) {
57             if (eq) return index;
58             if (isset) continue;
59         } else {
60             if (!isset) continue;
61             if (!eq) continue;
62         }
63         return index;
64     }
65     return ~0;
66 }
67
68 void *hashmap_get(struct hashmap *base, const void *key)
69 {
70     unsigned index = hashmap_keyindex(base, key, 0);
71     if (index == ~0u) return NULL;
72     return hashmap_slot_at(base, index);
73 }
74
75 void hashmap_resize(struct hashmap *base)
76 {
77     void *items;
78     unsigned char *bitmap;
79     size_t i, cap;
80     struct context *ctx;

```

```

81
82     items = base->items;
83     bitmap = base->bitmap;
84     cap = base->cap;
85     ctx = context_from_alloc(base);
86
87     if (base->cap) base->cap *= 2;
88     else base->cap = 1;
89
90     base->items = context_alloc(ctx, base->cap * base->itemsize);
91     base->bitmap = context_alloc(ctx, (base->cap + 7)>>3);
92     base->count = 0;
93
94     for (i = 0; i < cap; ++i) {
95         if (!(bitmap[i>>3]&(1<<(i&7)))) continue;
96         hashmap_insert(base, (char *)items + i*base->itemsize);
97     }
98
99     context_free(items);
100    context_free(bitmap);
101 }
102
103 int hashmap_insert(struct hashmap *base, const void *kv)
104 {
105     unsigned index;
106     int new;
107     for (;;) {
108         index = hashmap_keyindex(base, kv, 1);
109         if (index == ~0u) hashmap_resize(base);
110         else break;
111     }
112
113     new = !hashmap_index_ctl(base, index, ICTL_IS_SET);
114     hashmap_index_ctl(base, index, ICTL_SET);
115     base->count += new;
116     memcpy(hashmap_slot_at(base, index), kv, base->itemsize);
117     return new;
118 }
119
120 void *hashmap_next(struct hashmap *base, size_t *iter)
121 {
122     for (;;) (*iter)++ {
123         if (*iter >= base->cap) return NULL;
124         if (hashmap_index_ctl(base, *iter, ICTL_IS_SET))
125             return hashmap_slot_at(base, (*iter)++);
126     }
127 }
128
129 size_t hashmap_get_count(struct hashmap *base)
130 {
131     return base->count;
132 }
133
134 unsigned fnv1(const void *bytes, size_t n)
135 {
136     unsigned fp = 16777619;
137     unsigned hash = 2166136261;
138     const char *b = bytes;

```

```

139     const char *end = b + n;
140
141     for (; b != end; ++b) {
142         hash ^= *b;
143         hash *= fp;
144     }
145     return hash;
146 }

```

6 Выводы

В ходе выполнения практической работы была разработана консольная программа, моделирующая работу небольшой службы доставки. Программа полностью работает в оперативной памяти, не использует внешних файлов, баз данных и сетевых запросов, и реализована на языке C в соответствии со стандартом C89.

Реализованная функциональность

В соответствии с заданием были реализованы все обязательные сценарии работы:

- создание нового заказа с добавлением одной или нескольких позиций;
- просмотр списка существующих заказов;
- выбор способа доставки с автоматическим расчётом стоимости и ориентировочного срока;
- назначение курьера для тех способов доставки, которые его предусматривают;
- перевод заказа по допустимым статусам с проверкой корректности перехода;
- завершение работы по явной команде пользователя (`quit`, `exit` или `Ctrl+D`).

Помимо этого, реализованы обработка ошибочного ввода, валидация полей клиента (телефон, электронная почта) и демонстрационное наполнение реестров при старте программы, позволяющее сразу приступить к работе с уже существующими заказами.

Соблюдение принципов объектно-ориентированного программирования

Несмотря на то что язык C не поддерживает классы и интерфейсы непосредственно, все ключевые принципы ООП были воспроизведены средствами языка:

1. **Инкапсуляция.** Состояние сущностей (`order`, `client_data`, `courier`, `destination_address`, `order_service`) скрыто за непрозрачными указателями; изменение состояния выполняется только через функции-методы, которые проверяют допустимость операции. Например, изменение статуса заказа возможно только через `order_change_status`, которая сначала спрашивает у стратегии доставки, разрешён ли такой переход.
2. **Композиция.** Заказ содержит клиента, адрес доставки, массив позиций и выбранный способ доставки; сервис заказов содержит реестры клиентов и курьеров, а также массив заказов. Иерархия владения отражена в диаграмме классов.
3. **Полиморфизм.** Три способа доставки (*Standard*, *Express*, *Self pickup*) реализованы как три конкретные стратегии одного абстрактного интерфейса `order_kind`. Общий контракт описан набором методов (`calc_cost`, `calc_estimated_time`, `needs_courier`, `allows_transition`), а вызовы идут через таблицу виртуальных методов. Благодаря этому код, оформляющий заказ, не содержит проверок типа доставки — каждая стратегия сама знает, как считать стоимость, сколько времени займёт доставка и нужен ли ей курьер.
4. **Разделение ответственности.** Программа разделена на три слоя: `runtime` (управление памятью), `service` (бизнес-логика) и `tui` (взаимодействие с пользователем). Слой сервисов ничего не знает о консоли, слой TUI ничего не знает о деталях хранения. Связь между ними осуществляется через один указатель `userdata`.

Принятые архитектурные решения

В ходе работы были приняты следующие технические решения, обоснованные требованиями задания и выбранным языком:

- **Арена-аллокатор.** Вся память выделяется в едином контексте (`context_alloc`) и освобождается централизованно при завершении программы или при выходе из подконтекста. Это снимает необходимость следить за временем жизни каждого объекта вручную и позволяет сосредоточиться на бизнес-логике.
- **Таблица виртуальных методов.** Для реализации полиморфизма способов доставки использован классический приём — структура с указателем на таблицу функций.

Перечень методов описан макросом `ORDER_KIND_METHODS`, что обеспечивает единый источник правды: при добавлении нового метода компилятор заставляет реализовать его во всех стратегиях.

- **Собственная хеш-таблица.** Для реестров клиентов и курьеров реализована открытая адресация с битовой картой занятости слотов. Контейнер компактен и достаточен для нагрузки, характерной для данной задачи.
- **Стек меню.** Интерфейс пользователя построен как стек состояний: каждое меню имеет свой набор команд, переход в подменю и возврат выполняются операциями `menu_push_state` и `menu_pop_state`. Такая схема позволяет, например, из меню создания заказа перейти в меню создания клиента, зарегистрировать его и вернуться к тому же черновику заказа, не потеряв введённые данные.
- **Граф допустимых переходов статусов.** Каждый способ доставки описывает свои переходы в виде таблицы пар «откуда–куда». Это делает правила прозрачными и позволяет добавлять новые способы доставки без правки общего кода смены статуса.

Полученные навыки

В процессе выполнения работы были отработаны:

- проектирование объектной модели предметной области и распределение ответственности между слоями приложения;
- реализация полиморфизма средствами языка без встроенной поддержки классов;
- организация управления памятью с использованием арена-аллокатора и явных границ владения объектами;
- построение конечного автомата состояний и вынесение правил переходов в отдельные сущности;
- разработка интерактивного консольного интерфейса с поддержкой стека меню, подсказок и обработки ошибочного ввода;
- работа с системой контроля версий Git и оформление проекта в виде отдельного репозитория.

Заключение

Поставленная задача выполнена в полном объёме. Программа функционирует согласно требованиям задания, поддерживает все заявленные сценарии работы и корректно обрабатывает ошибочные ситуации. Использование языка С потребовало самостоятельной реализации ряда механизмов, обычно предоставляемых языками высокого уровня (управление памятью, полиморфизм, контейнеры), что позволило глубже понять внутреннее устройство соответствующих абстракций. Полученный код отделён от конкретного способа взаимодействия с пользователем и от конкретного набора способов доставки, что делает его расширяемым: добавление четвёртого способа доставки сводится к созданию одного нового файла без правок в существующей логике.